

Single Sign-On, OpenID Connect & iDM

Memahami Manajemen Identitas, Otentikasi & iDM

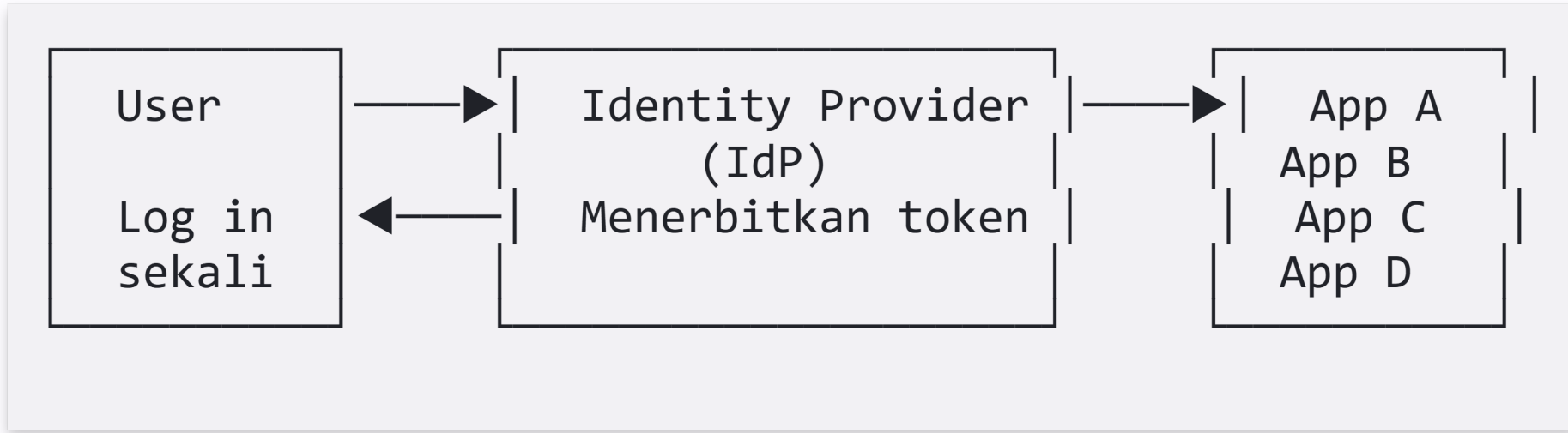
Daftar Isi

1. Single Sign-On (SSO) — Apa Itu dan Mengapa Penting
2. OpenID Connect (OIDC) — Protokol di Balik SSO Modern
3. Manajemen Identitas (iDM)

Bagian 1: Single Sign-On (SSO)

Apa Itu SSO?

Single Sign-On (SSO) adalah skema otentikasi yang memungkinkan pengguna untuk masuk satu kali dengan satu kredensial dan mendapatkan akses ke beberapa aplikasi atau layanan independen tanpa harus memasukkan kembali kredensial.



Mengapa SSO Penting

Manfaat	Deskripsi
Pengalaman Pengguna	Satu login → akses ke semua. Tidak ada lagi kelelahan kata sandi.
Keamanan	Lebih sedikit kata sandi = permukaan serangan berkurang. Penegakan MFA terpusat.

Manfaat	Deskripsi
Administrasi	Manajemen identitas terpusat. Cukup cabut akses sekali, berlaku di mana-mana.
Kepatuhan	Jejak audit untuk siapa mengakses apa, terpadu di seluruh layanan.
Produktivitas	Lebih sedikit waktu untuk mengatur ulang kata sandi dan membuka kunci akun.

Cara Kerja SSO (Alur Konseptual)

1. Pengguna —► Browser membuka App A
2. App A —► Mengarahkan ke Identity Provider (IdP)
3. Pengguna —► Melakukan otentikasi dengan IdP (kata sandi, passkey, MFA)
4. IdP —► Menerbitkan token/assertion kembali ke browser
5. Browser —► Menyajikan token ke App A
6. App A —► Memvalidasi token dengan IdP → memberikan akses
7. Pengguna membuka App B —► Sudah terotentikasi → akses tanpa hambatan

Protokol utama SSO:

- SAML 2.0 — Berbasis XML, standar warisan perusahaan
- OIDC (OpenID Connect) — Modern, berbasis REST/JSON, dibangun di atas OAuth 2.0
- Kerberos — Berbasis tiket, umum di Windows Active Directory

Tantangan SSO

- Titik tunggal kegagalan — Jika IdP mati, tidak ada yang bisa masuk ke mana pun
- Keterikatan vendor — Implementasi SSO proprietary sulit untuk dipindahkan
- Kompleksitas — Menyiapkan hubungan kepercayaan, manajemen sertifikat, pertukaran metadata
- Integrasi — Aplikasi lama mungkin tidak mendukung protokol modern (perlu jembatan LDAP/warisan)

- Trade-off keamanan — Sesi SSO yang dikompromikan = akses ke semuanya (diminimalisir dengan MFA, otentikasi bertingkat, token berumur pendek)

Bagian 2: OpenID Connect (OIDC)

Apa Itu OIDC?

OpenID Connect (OIDC) adalah lapisan identitas sederhana di atas OAuth 2.0. OIDC memungkinkan klien untuk memverifikasi identitas pengguna akhir berdasarkan otentikasi yang dilakukan oleh Authorization Server, serta mendapatkan informasi profil dasar tentang pengguna tersebut.



- Diterbitkan sebagai spesifikasi pada 2014 oleh OpenID Foundation
- Menggunakan JSON Web Tokens (JWT) untuk assertion identitas
- Dirancang untuk klien web, mobile, dan API

Konsep Inti OIDC

Konsep	Deskripsi
OpenID Provider (OP)	Server yang mengotentikasi pengguna dan menerbitkan token (mis. Kanidm, Keycloak, Google)
Relying Party (RP)	Aplikasi yang mengandalkan OP untuk otentikasi

Konsep	Deskripsi
ID Token	JWT yang berisi klaim tentang pengguna yang terotentikasi (sub, email, nama, dll.)
Access Token	Token yang digunakan untuk mengakses API atas nama pengguna (dari OAuth 2.0)
Claims	Pasangan kunci-nilai dalam token yang menjelaskan pengguna (mis. sub, email, groups)

Konsep

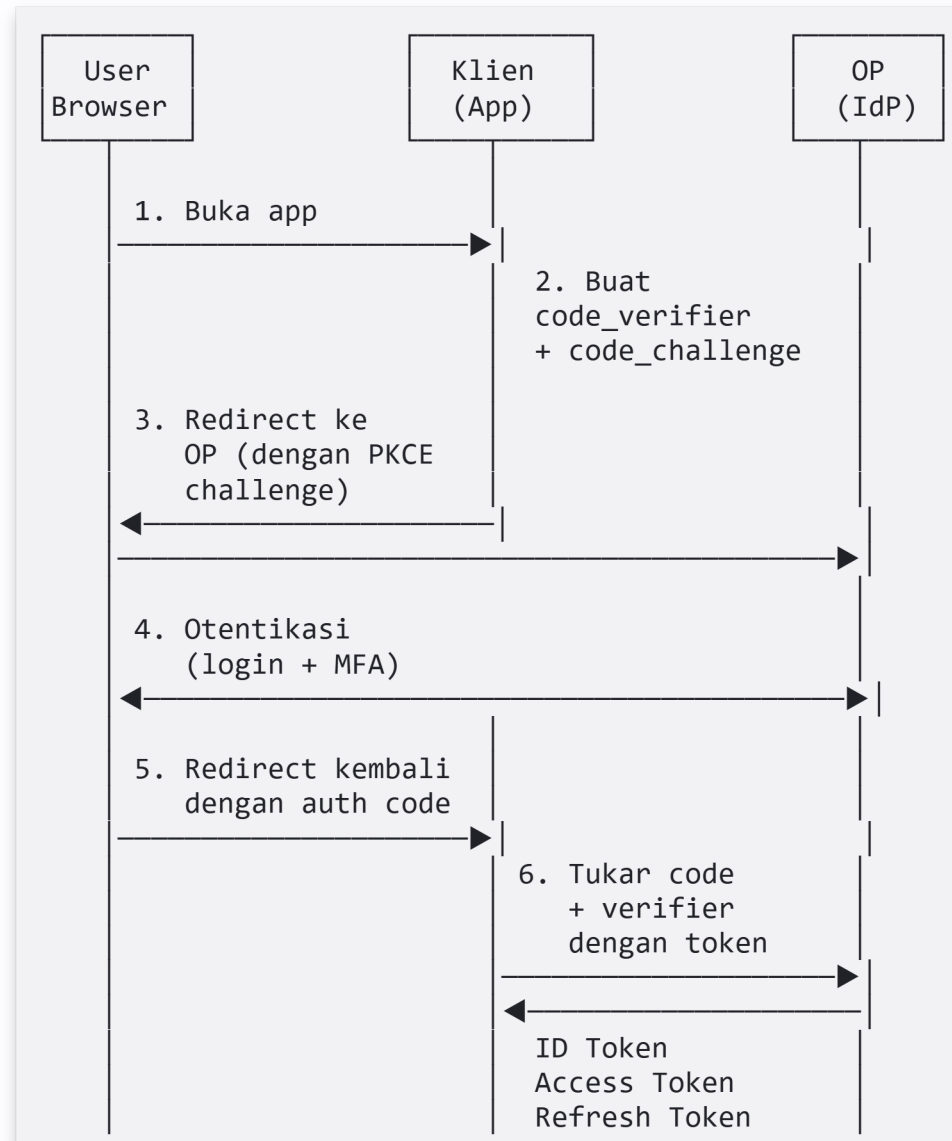
Deskripsi

Scopes

Izin yang diminta oleh klien (mis. openid, profile, email, groups)

Alur Authorization Code OIDC dengan PKCE

Alur yang paling direkomendasikan untuk aplikasi web dan mobile:



PKCE (Proof Key for Code Exchange) mencegah serangan injeksi kode otorisasi. PKCE sekarang dianggap wajib untuk semua klien OIDC.

Contoh ID Token (JWT)

ID Token OIDC yang didekode berisi tiga segmen berenkode base64 (header, payload, tanda tangan):

```
// Header
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "kanidm-rsa-key-1"
}

// Payload (Klaim)
{
  "iss": "https://idp.example.com/auth/realms/main",
  "sub": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "aud": "my-web-app",
  "exp": 1735689600,
  "iat": 1735686000,
  "auth_time": 1735685995,
  "email": "astra@example.com",
  "email_verified": true,
  "name": "Astra",
  "preferred_username": "astra",
  "groups": ["developers", "admin"]
}
```

Endpoint OIDC

Endpoint	Tujuan
/authorize	Otentikasi & otorisasi pengguna (redirect browser)
/token	Menukar kode otorisasi dengan token
/userinfo	Mendapatkan klaim pengguna tambahan (optional — ID token mungkin sudah cukup)

Endpoint	Tujuan
/authorize	Otentikasi & otorisasi pengguna (redirect browser)
/jwks	Kunci publik untuk memverifikasi tanda tangan token
/.well-known/openid-configuration	Dokumen discovery OIDC (mendaftar semua endpoint, scope yang didukung, dll.)

OIDC vs SAML vs OAuth 2.0

Fitur	SAML 2.0	OAuth 2.0	OIDC
Tujuan	Otentikasi (SSO)	Otorisasi (akses API)	Otentikasi + Identitas
Format	XML	Query params / JSON	JSON / JWT

Fitur	SAML 2.0	OAuth 2.0	OIDC
Token	SAML Assertions	Access Token	ID Token + Access Token
Ramah mobile	✗ Kurang	✓ Ya	✓ Ya
Adopsi modern	Warisan perusahaan	Universal	Berkembang pesat

Fitur	SAML 2.0	OAuth 2.0	OIDC
Dibangun di atas	Warisan XML/SOAP	—	OAuth 2.0

OIDC = OAuth 2.0 (otorisasi) + Lapisan Identitas (otentikasi)

Bagian 3: Manajemen Identitas (iDM)

Apa itu iDM?

Mengonsolidasikan cara organisasi:

- mengelola identitas (user, layanan, perangkat)
- melakukan autentikasi (siapa yang login)
- melakukan otorisasi (siapa yang berhak akses)
- melakukan audit dan tata kelola akses

Kenapa IdM Penting?

- Mengurangi “password sprawl” (banyak password berbeda)
- Memperkuat keamanan (MFA, validasi terpusat)
- Memudahkan SSO (Single Sign-On)
- Standarisasi kebijakan akses antar aplikasi
- Menambah visibilitas untuk kepatuhan dan investigasi insiden
- Mengotomasi provisioning & deprovisioning

Komponen Utama IdM

- Identity store / direktori: pengguna, grup, atribut
- Autentikasi: verifikasi identitas
- Otorisasi: keputusan akses (izin/role)
- Provisioning: membuat/mengubah entitas & peran
- Auditing: pencatatan jejak aktivitas
- Governance: kebijakan, persetujuan, dan lifecycle

Siklus Hidup Identitas

- Membuat identitas (karyawan, kontraktor, service account)
- Menetapkan atribut (departemen, role, level akses)
- Memberi akses (role aplikasi, keanggotaan grup)
- Mengubah akses sesuai perubahan kebutuhan
- Menonaktifkan/deprovision saat offboarding
- Menjaga audit dan riwayat perubahan

Metode Autentikasi

- Password (umumnya warisan/legacy)
- MFA (Multi-Factor Authentication)
- Sertifikat (untuk service/device)
- Federasi (OIDC/SAML)
- Risk-based/step-up authentication (berbasis risiko)

Apa Itu Kanidm?

Kanidm adalah platform manajemen identitas sederhana, aman, dan cepat yang ditulis dengan Rust.

 IDM yang di-hosting sendiri dengan provider
SSO/OIDC bawaan

 Dukungan WebAuthn/Passkey native (tanpa perlu
kata sandi!)

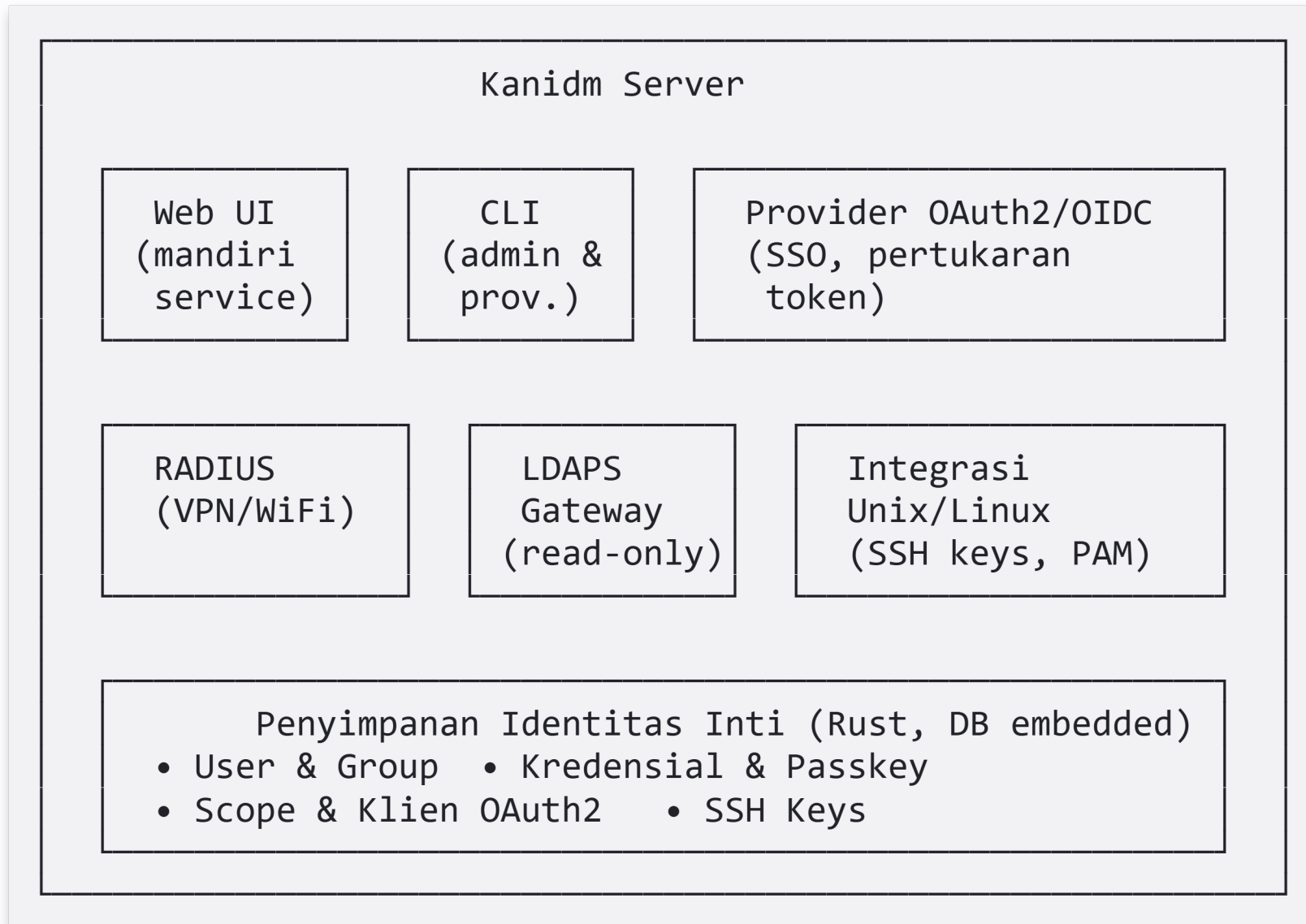
 Ditulis dalam Rust — memory-safe, performant,
ringan

 Berlisensi MPL 2.0 (open source)

 Dirancang sebagai alternatif untuk FreeIPA,
kombinasi Keycloak + LDAP

 Versi saat ini: v1.11.x

Arsitektur Kanidm



Fitur Utama Kanidm

Otentikasi

- ✓ Passkey WebAuthn – otentikasi kriptografis, tahan phishing
- ✓ Passkey yang diattestasi untuk lingkungan keamanan tinggi
- ✓ Opsi TOTP otentikasi dua faktor
- ✓ PKCE ditegakkan secara default pada semua alur OIDC/OAuth2

Manajemen Identitas

- ✓ Provider OAuth2/OIDC bawaan – tidak perlu portal eksternal
- ✓ Portal Aplikasi – pengguna hanya melihat aplikasi yang dapat mereka akses
- ✓ Gateway LDAPS read-only untuk integrasi LDAP warisan
- ✓ Dukungan RADIUS untuk otentikasi jaringan/VPN
- ✓ Tooling CLI lengkap untuk administrasi dan provisioning

Integrasi Linux/Unix

- ✓ Otentikasi offline yang dilindungi TPM
- ✓ Distribusi kunci SSH ke server
- ✓ Integrasi PAM/NSS

Prinsip Desain Kanidm

Prinsip	Bagaimana Kanidm Mengimplementasikannya
---------	--

Aman
secara
default

PKCE ditegakkan, 2FA diwajibkan sejak awal, sesi persisten telah dikonfigurasi

Pemisahan
hak
istimewa

Admin sistem dan admin IDM adalah peran terpisah — keduanya tidak dapat memulihkan kata sandi satu sama lain

Prinsip

Bagaimana Kanidm Mengimplementasikannya

Otentikasi
berbasis
perangkat

Pengguna melakukan otentikasi dengan perangkat; perangkat menyimpan kredensial ter-scoped untuk setiap layanan

Provisioning
deklaratif

User, group, dan klien OIDC dapat didefinisikan dalam konfigurasi (mis. Nix) untuk deployment yang dapat direproduksi

Prinsip

Bagaimana Kanidm Mengimplementasikannya

Dukungan
UTF-8 penuh

Nama tampilan mendukung emoji dan karakter Unicode apa pun

Jejak minimal

Satu biner Rust, tanpa JVM, tanpa database eksternal yang diperlukan

Kapan Memilih Kanidm

✓ Sangat cocok jika Anda:

- Menginginkan IDM self-hosted yang ringan tanpa JVM
- Membutuhkan SSO OIDC/OAuth2 bawaan (tanpa Keycloak terpisah)
- Menghargai WebAuthn/passkey sebagai metode otentikasi utama
- Mengelola infrastruktur Linux/Unix (kunci SSH, PAM)
- Membutuhkan RADIUS untuk otentikasi VPN/WiFi
- Menghargai provisioning deklaratif yang dapat direproduksi (Nix, Ansible, dll.)
- Menginginkan default keamanan yang kuat (PKCE diwajibkan, 2FA diwajibkan, pemisahan hak istimewa)

✗ Mungkin bukan pilihan terbaik jika Anda:

- Membutuhkan LDAP yang dapat ditulis penuh (Kanidm hanya menyediakan LDAPS read-only)
- Membutuhkan dukungan SAML (Kanidm fokus pada OIDC)
- Memiliki kebutuhan federasi perusahaan yang kompleks lintas organisasi
- Membutuhkan alur kerja administrasi berbasis GUI yang ekstensif

Sumber Lanjutan

- Dokumentasi Kanidm: kanidm.github.io/kanidm
- Website Kanidm: kanidm.com
- GitHub Kanidm: github.com/kanidm/kanidm
- Spesifikasi OIDC: openid.net/specs/openid-connect-core-1_0.html
- OAuth 2.0 PKCE (RFC 7636):
[9datatracker.ietf.org/doc/html/rfc7636](https://datatracker.ietf.org/doc/html/rfc7636)
- Perbandingan Kanidm: kanidm.com/comparisons

Instalasi dan Konfigurasi KaniDM